

Comparing Reinforcement Learning Methods in Simulated Mean-Reverting Stock Trading

Abstract

This project studies the application of reinforcement learning to a simulated mean-reverting stock trading problem. The trading process is formulated as a continuous-action Markov Decision Process. At each time step, the agent observes the market state, chooses a position A_t , and receives a reward based on trading profit and regularization or transaction-cost penalties. The project compares three common continuous-control reinforcement learning algorithms: PPO, TD3, and SAC.

The stock price is generated by a simulated mean-reverting log-return process. Two reward functions are considered. Reward (1) penalizes absolute position size, while Reward (2) penalizes changes in position and therefore approximates transaction costs. For Reward (1), the optimal policy can be derived analytically because the action does not affect future price dynamics. This analytic policy is used as a benchmark for evaluating learned policies.

The high budget experiment uses 1000 independent Monte Carlo evaluation paths with evaluation horizon 80. The results show that TD3 and SAC generally outperform PPO. Under the baseline R1 oracle setting, SAC achieves a mean return of 485.16 and TD3 achieves 479.98, corresponding to 52.27% and 51.71% of the analytic benchmark. PPO obtains a mean return of -158.29. Under the baseline R2 oracle setting, SAC obtains a mean return of 464.66, slightly above TD3's 453.52, but SAC has a turnover of 6.98 compared with TD3's 2.80, indicating a more aggressive trading policy. Under the baseline R2 sequential setting, TD3 performs best with a mean return of 620.20 and low turnover.

Overall, the experiment shows that reinforcement learning can naturally model trading as a sequential decision problem, but performance depends strongly on reward design, data protocol, and market parameters. TD3 is the most robust method in this experiment, SAC is competitive in oracle settings but tends to trade more aggressively, and PPO is more conservative but less sample-efficient.

1. Background

Financial trading is a sequential decision-making problem. A trader observes market information over time, adjusts positions repeatedly, and receives profits or losses from future price movements. Unlike one-step price prediction, trading strategy learning requires choosing a position that balances expected reward, risk exposure, and transaction costs over time.

Reinforcement learning provides a natural framework for this problem. In this project:

- The state represents observable market information.
- The action is the current trading position.
- The reward combines trading profit with regularization or transaction-cost penalties.
- The policy maps market states to trading actions.
- The objective is to maximize expected discounted total reward.

This project uses simulated data rather than real stock data. This choice makes the experiment more controlled. Market parameters such as mean-reversion speed, volatility, and penalty strength can be changed

directly. Independent evaluation paths can also be generated from the simulator. Most importantly, Reward (1) admits an analytic benchmark, which helps verify whether a learned policy captures the correct structure.

The goal is not to build a deployable trading strategy. Instead, the project demonstrates how a financial trading task can be formalized as a reinforcement learning problem and how different RL algorithms behave under the same simulated environment.

2. Market Model

Let S_t be the stock price with initial value $S_0 = 1$. The log-return process L_t follows:

$$L_{t+1} = (1 - \kappa) L_t + \sigma Z_t, \quad Z_t \sim N(0, 1)$$

$$S_{t+1} = S_t \exp(L_{t+1})$$

Here, $\kappa \in (0, 1)$ controls the speed of mean reversion, $\sigma > 0$ controls volatility, and Z_t is an independent standard normal random variable. Larger κ makes returns revert to zero faster, while larger σ creates more volatile price paths.

The action A_t is the number of shares held at time t . Positive values represent long positions, negative values represent short positions, and zero represents no position. Since A_t is continuous, this is a continuous-control problem. In implementation, actions are clipped to a bounded interval to avoid unstable extreme positions.

3. MDP Formulation

For Reward (1), the state is:

$$s_t = (\log S_t, L_t)$$

For Reward (2), the state is:

$$s_t = (\log S_t, L_t, A_{t-1})$$

Reward (2) must include the previous action A_{t-1} , because its cost term depends on the position change $A_t - A_{t-1}$. Without A_{t-1} , the agent cannot infer the cost of changing positions from the current state alone, and the Markov property would be violated.

The objective is:

$$\text{maximize } \mathbb{E} \left[\sum_t \gamma^t R_t \right]$$

The **mean_return** reported in this paper is the Monte Carlo estimate of this discounted total reward. It is not a stock return percentage.

4. Reward Design

4.1 Reward (1): Position Regularization

Reward (1) is:

$$R_t = A_t (S_{t+1} - S_t) - \lambda A_t^2$$

The first term is trading profit. A long position gains when the price rises, and a short position gains when the price falls. The second term penalizes large absolute positions, preventing the policy from taking unlimited leverage.

The key advantage of Reward (1) is analytical tractability. Since the action does not affect future price dynamics, the optimal action can be solved period by period. This analytic policy provides a benchmark for learned policies.

4.2 Reward (2): Position-Change Penalty

Reward (2) is:

$$R_t = A_t (S_{t+1} - S_t) - \lambda (A_t - A_{t-1})^2 S_t$$

Unlike Reward (1), this reward penalizes changes in position rather than the position size itself. It is closer to a stylized transaction-cost model, because frequent trading generates costs in real markets.

Under Reward (2), a reasonable policy should be smoother. If the previous position is already large, the agent should avoid making unnecessary large changes unless the market signal is strong enough.

5. Data Protocols

5.1 Simulation Oracle

In the simulation oracle setting, the algorithm can repeatedly generate new independent episodes from the known simulator. This is an idealized setting, similar to having unlimited access to a market simulator. It provides broader data coverage and generally makes training easier.

5.2 Sequential Data

In the sequential data setting, training data comes from a fixed price path. This is closer to historical financial data, where one cannot generate unlimited independent market trajectories. The data are more correlated and less diverse, making training more challenging.

6. Reinforcement Learning Algorithms

6.1 PPO

PPO, or Proximal Policy Optimization, is an on-policy policy-gradient method. It collects rollouts using the current policy and updates the policy using advantage estimates. Its core idea is the clipped objective:

$$\min \left(r_t A_t, \operatorname{clip}(r_t, 1 - \epsilon, 1 + \epsilon) A_t \right)$$

where r_t is the probability ratio between the new and old policies. The clipping mechanism prevents overly large policy updates and improves training stability.

In this project, PPO uses an actor network to output a continuous action distribution and a critic network to estimate the value function. PPO is stable but sample-inefficient because rollout data are mainly used for the current policy update and are not reused extensively.

In the high budget results, PPO is generally conservative. It has low turnover and smaller average positions, but its mean return is weak. Under the baseline R1 oracle setting, PPO obtains a mean return of -158.29,

indicating that it fails to learn an effective trading policy in this setup.

6.2 TD3

TD3, or Twin Delayed Deep Deterministic Policy Gradient, is an off-policy deterministic actor-critic method. The actor outputs a deterministic action $A_t = \mu(s_t)$, and the critics estimate $Q(s_t, A_t)$.

TD3 has three important mechanisms:

- 1. Twin critics: two Q networks are trained, and the smaller target value is used to reduce overestimation.
- 2. Delayed policy updates: the actor is updated less frequently than the critics, making actor updates more reliable.
- 3. Target policy smoothing: clipped noise is added to target actions to reduce overfitting to sharp Q-function peaks.

TD3 also uses a replay buffer, which allows it to reuse past transitions and improve sample efficiency. In the high budget results, TD3 is especially robust under sequential data. It achieves a mean return of 601.52 under baseline R1 sequential and 620.20 under baseline R2 sequential, both the best results in their settings.

6.3 SAC

SAC, or Soft Actor-Critic, is an off-policy stochastic actor-critic method. It adds an entropy term to the reward objective:

$$\mathbb{E}[\text{expected reward}] + \alpha \times \mathbb{E}[\text{entropy}]$$

This encourages exploration and prevents the policy from becoming deterministic too quickly. In noisy financial environments, this entropy regularization can help the policy explore a wider range of trading behaviors.

In the high budget results, SAC is strong in oracle settings. Under baseline R1 oracle, SAC obtains a mean return of 485.16, slightly above TD3. Under baseline R2 oracle, SAC obtains a mean return of 464.66, also slightly above TD3. However, SAC has a turnover of 6.98 under R2 oracle, much higher than TD3's 2.80, suggesting a more aggressive trading policy.

7. Experiment Setup

This report uses the results in [output_highbudget/results_main.csv](#).

Item	Setting
Parameter sets	baseline, slow_reversion, high_volatility, strong_regularization
Seeds	0, 1
Rewards	R1, R2
Data settings	simulation oracle, sequential
Algorithms	PPO, TD3, SAC
Monte Carlo evaluation paths	1000

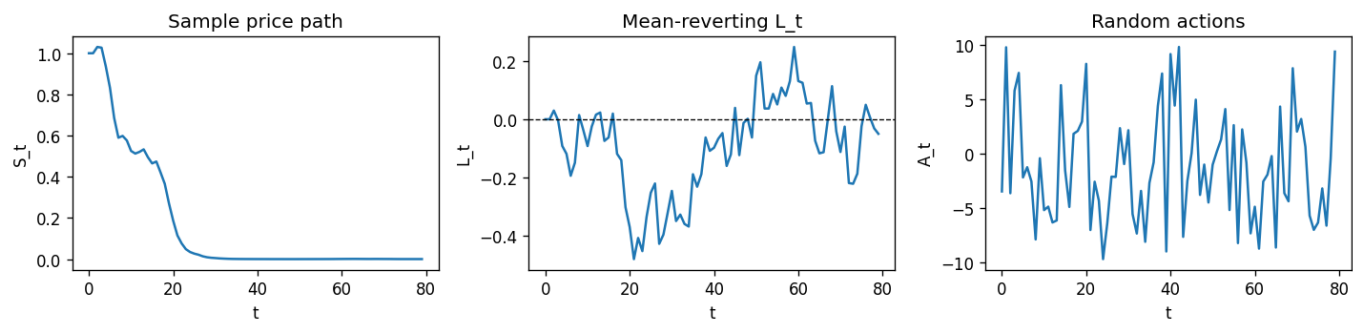
Item	Setting
Evaluation horizon	80

The main metrics are:

- `mean_return`: mean discounted total reward.
- `std_error`: Monte Carlo standard error.
- `turnover`: average position change.
- `avg_abs_position`: average absolute position.
- `relative_to_analytic_pct`: learned policy return as a percentage of the Reward (1) analytic benchmark.
- `policy_mse_to_analytic`: mean squared error between a learned Reward (1) policy and the analytic policy.

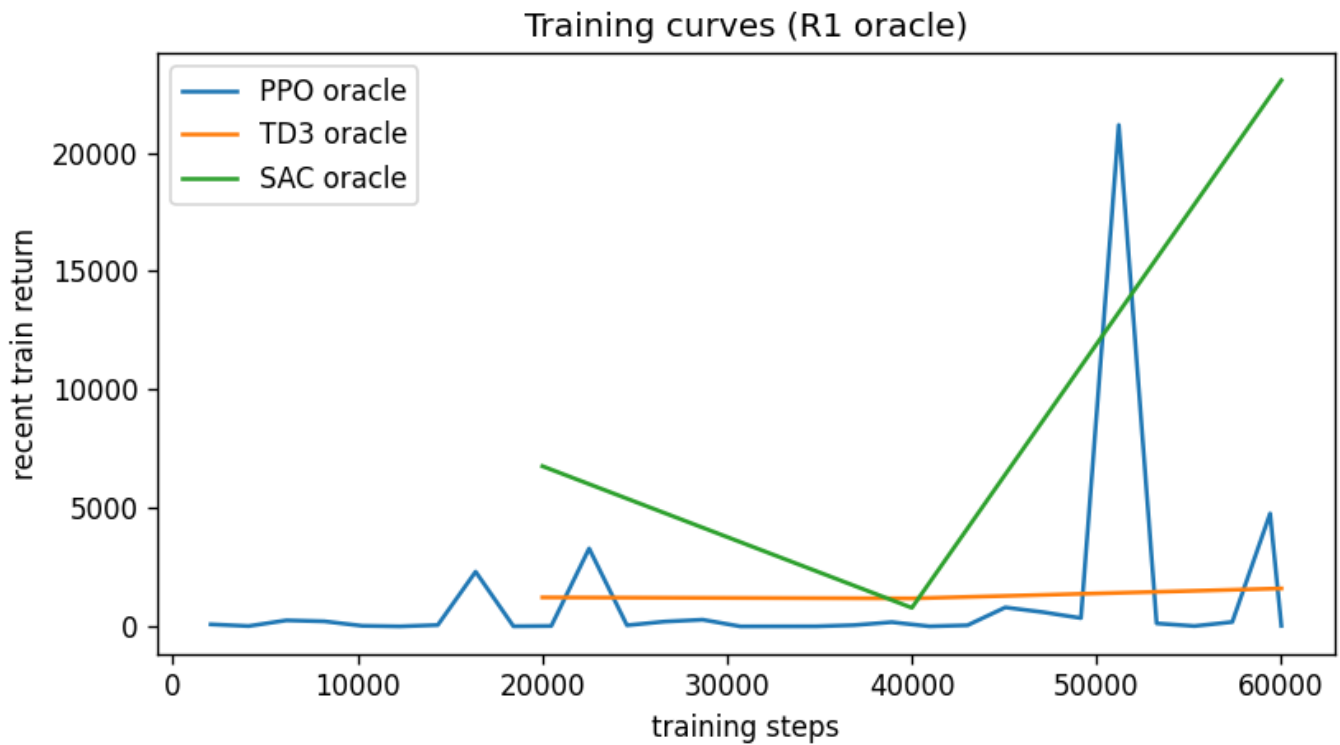
8. Figure Interpretation

8.1 Environment Sanity Check



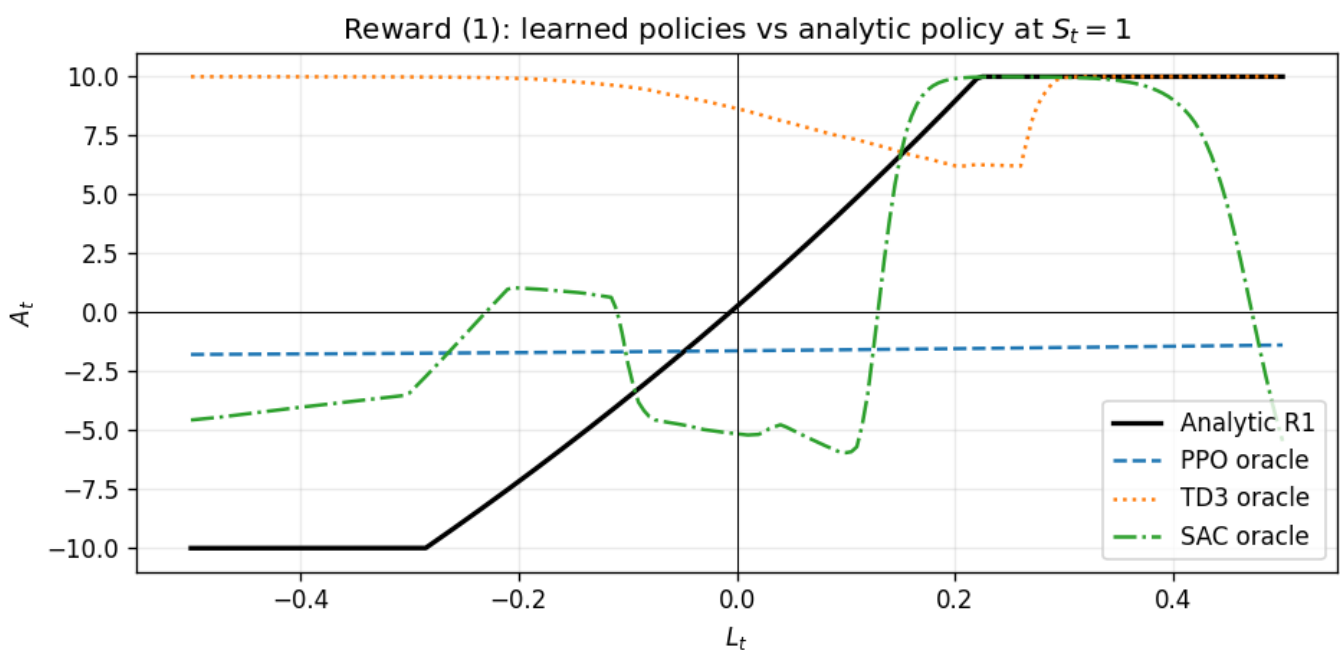
This figure verifies that the environment is working correctly. It shows a simulated price path, the mean-reverting return process, and a random action trajectory. The purpose is not performance evaluation, but checking that the simulator and action clipping behave reasonably.

8.2 Training Curves



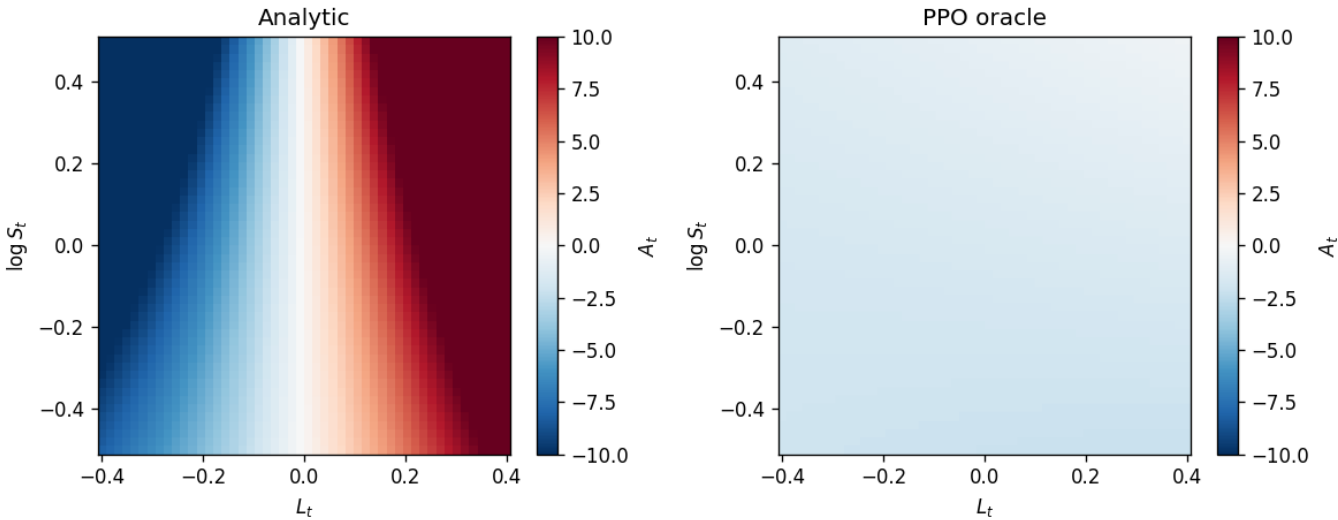
The training curves show PPO, TD3, and SAC under the baseline R1 oracle setting. They help identify whether training is stable or collapsing. However, the final comparison should rely on independent Monte Carlo evaluation rather than training rewards alone.

8.3 Reward (1) Policy Comparison



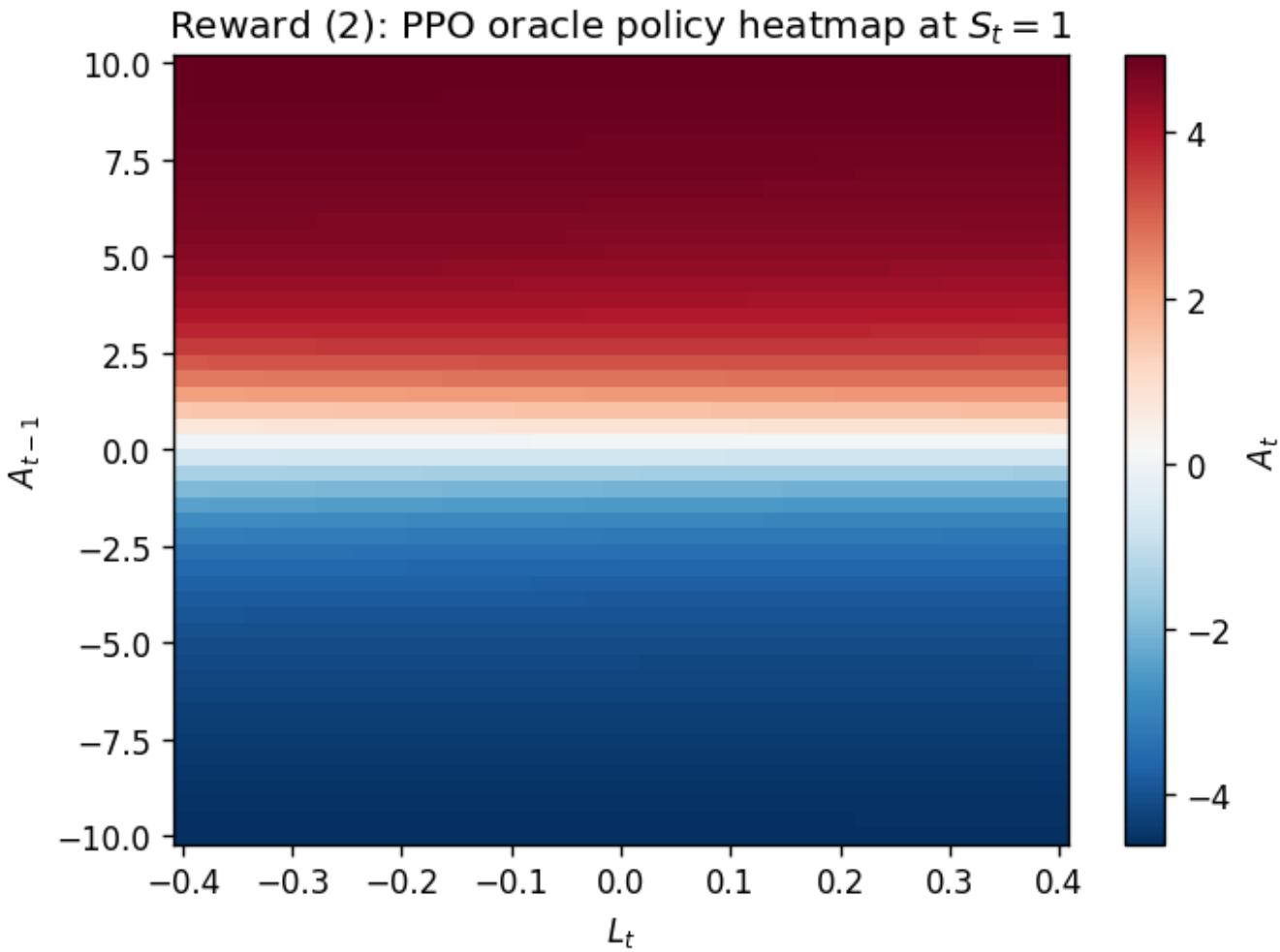
This figure compares learned policies with the analytic Reward (1) policy. A learned policy closer to the analytic curve is more likely to have captured the correct mean-reversion trading structure. A flat curve suggests an overly conservative policy, while a curve stuck near the action boundary suggests an overly aggressive policy.

8.4 Reward (1) Policy Heatmap



The heatmap shows the action selected for different values of $\log S_t$ and L_t . Smooth color changes indicate stable responses to state changes. Large regions of extreme color may indicate that the policy often takes boundary positions.

8.5 Reward (2) Policy Heatmap



For Reward (2), the policy depends on the previous position A_{t-1} . This heatmap helps show whether the policy has inertia under transaction-cost-like penalties. A reasonable policy should avoid unnecessary large jumps between positions.

9. Baseline Results

The following table averages the two seeds under the baseline parameter set.

Reward	Setting	Method	Mean return	Std. error	Turnover	Avg. abs. position	Relative to analytic	Policy MSE
R1	oracle	SAC	485.16	192.69	1.66	5.72	52.27%	25.29
R1	oracle	TD3	479.98	177.91	1.39	4.61	51.71%	85.64
R1	oracle	PPO	-158.29	79.53	0.18	2.62	-17.05%	65.40
R1	sequential	TD3	601.52	212.87	0.13	10.00	64.80%	160.91
R1	sequential	PPO	-86.47	60.74	0.56	1.34	-9.32%	37.41
R1	sequential	SAC	-135.90	199.27	0.37	4.70	-14.64%	132.93
R2	oracle	SAC	464.66	202.54	6.98	6.41	N/A	N/A
R2	oracle	TD3	453.52	174.68	2.80	4.96	N/A	N/A
R2	oracle	PPO	-36.66	113.17	0.18	3.88	N/A	N/A
R2	sequential	TD3	620.20	212.87	0.16	9.34	N/A	N/A
R2	sequential	PPO	-195.06	76.71	0.41	2.43	N/A	N/A
R2	sequential	SAC	-222.96	210.86	2.33	7.09	N/A	N/A

9.1 R1 Oracle

R1 oracle is the cleanest setting for comparing learning ability because it has an analytic benchmark. SAC and TD3 perform similarly, reaching 52.27% and 51.71% of the analytic benchmark. PPO has a negative mean return, showing that it does not exploit the mean-reversion signal effectively.

SAC has a policy MSE of 25.29, lower than TD3's 85.64, suggesting that SAC's policy shape is closer to the analytic policy. TD3 has similar return but a larger policy-shape deviation.

9.2 R1 Sequential

Under R1 sequential, TD3 performs best with a mean return of 601.52 and turnover of only 0.13. PPO and SAC both have negative returns. This indicates that TD3 is more robust when training data come from a fixed sequential path.

9.3 R2 Oracle

Under R2 oracle, SAC has the highest mean return, 464.66, slightly above TD3's 453.52. However, SAC's turnover is 6.98, much higher than TD3's 2.80 and PPO's 0.18. This suggests that SAC achieves high reward with more frequent position changes.

9.4 R2 Sequential

Under R2 sequential, TD3 is clearly best, with mean return 620.20 and turnover 0.16. PPO and SAC are both negative. This shows that TD3 is the most robust method when fixed sequential data and transaction-cost-like penalties are both present.

10. Parameter-Set Comparison

The following table summarizes R1 oracle performance across parameter sets.

Parameter set	Method	Mean return	Relative to analytic	Turnover	Policy MSE
baseline	SAC	485.16	52.27%	1.66	25.29
baseline	TD3	479.98	51.71%	1.39	85.64
baseline	PPO	-158.29	-17.05%	0.18	65.40
high_volatility	SAC	1.250e+08	58.67%	0.47	152.27
high_volatility	PPO	-9.820e+06	-4.61%	0.35	74.73
high_volatility	TD3	-1.359e+07	-6.38%	0.41	166.72
slow_reversion	TD3	5.868e+15	55.76%	0.44	106.65
slow_reversion	SAC	5.863e+15	55.71%	0.83	81.61
slow_reversion	PPO	-9.220e+14	-8.76%	0.11	74.08
strong_regularization	TD3	477.50	52.32%	1.24	22.86
strong_regularization	SAC	397.42	43.54%	1.60	56.66
strong_regularization	PPO	181.68	19.91%	0.16	5.53

The table shows that performance is sensitive to market parameters. TD3 and SAC dominate PPO under baseline, slow_reversion, and strong_regularization settings. Under high_volatility, SAC performs best, while PPO and TD3 are negative. This indicates that high volatility creates a more difficult training regime and amplifies algorithmic differences.

11. Analytic Benchmark

The Reward (1) analytic benchmark is:

Parameter set	Analytic mean return	Std. error	Turnover	Avg. abs. position
baseline	928.21	334.39	2.48	4.92
slow_reversion	1.052e+16	1.045e+16	1.45	5.36
high_volatility	2.131e+08	1.652e+08	2.68	5.61
strong_regularization	912.69	334.31	1.44	2.86

The slow_reversion and high_volatility settings produce very large values because the price model uses $S_t = \exp(\log S_t)$. When volatility is high or mean reversion is slow, extreme price paths can occur. These magnitudes should therefore be interpreted as model sensitivity rather than realistic trading profits.

12. Algorithm Comparison Summary

PPO is stable and conservative, but it is the weakest method in this experiment. Its turnover is usually low, but its mean return is often negative.

TD3 is the most robust method overall. It performs especially well under sequential data settings, suggesting that its replay buffer, twin critics, and deterministic actor are effective for fixed-path trading data.

SAC is competitive in oracle settings and slightly outperforms TD3 under baseline R1 oracle and R2 oracle. However, SAC can have high turnover, especially under R2 oracle, indicating more aggressive trading behavior.

Overall, TD3 is more robust, SAC explores more and performs well in oracle settings, and PPO is more conservative but less profitable under the current training setup.

13. Limitations and Future Work

This project uses a simulated market and omits many real-world trading frictions, including bid-ask spreads, slippage, market impact, capital constraints, and market nonstationarity. Therefore, the learned policies should not be interpreted as directly tradable strategies.

The experiment uses two training seeds. Although the high budget evaluation uses 1000 Monte Carlo paths, training randomness can still affect the conclusions. A more rigorous study should use more seeds and report confidence intervals.

The main metric is mean discounted total reward. Real investment strategies should also be evaluated using Sharpe ratio, maximum drawdown, tail risk, VaR/CVaR, and capital usage.

Some parameter sets produce extreme reward magnitudes, showing that the exponential price model can generate extreme paths under high volatility or slow mean reversion. Future work could add price clipping, log-return-based rewards, or risk-adjusted objectives.

Future extensions include using real stock or ETF data, applying walk-forward validation, adding realistic transaction costs, comparing more algorithms such as DDPG, A2C, or discrete-action DQN, and performing hyperparameter ablations over learning rate, network width, batch size, and replay buffer size.

14. Conclusion

This project demonstrates a complete reinforcement learning pipeline for a financial trading task: constructing a simulated market environment, defining states, actions, and rewards, implementing PPO, TD3, and SAC, evaluating learned policies through Monte Carlo simulation, and visualizing policy behavior.

The high budget results show that TD3 and SAC generally outperform PPO. SAC is competitive in oracle settings but has higher turnover, TD3 is more robust under sequential data, and PPO is conservative but weak in return.

The main contribution of the project is to clearly demonstrate how financial trading can be formalized as a reinforcement learning problem and how different RL algorithms learn different trading behaviors under the same environment.